Mês 5 — Extração Completa dos Descritores (jan/2027)

Bolsista PIBITI

Janeiro de 2027

Sumário

1 O que o plano exige 2

2 Conjunto completo de descritores 3

3 Entropia do sinal 4

4 Fator de forma 5

5 Fator de crista 6

6 Extrator completo 7

7 Gerando a matriz de features 8

8 Salvando e carregando features 9

9 Sanity checks 10

10 Análise exploratória com PCA 11

11 Cache de features 12

12 Brincadeiras 13

13 Checklist 14

14 Desafio final 15

Capítulo 1

O que o plano exige

Neste quinto mês do projeto PIBITI, o objetivo ê finalizar o conjunto de descritores de contorno de onda e construir o vetor de características completo para cada sinal de âudio, utilizando a Wavelet Packet Transform (WPT).

Marco / Entrega

Objetivo: Implementar o conjunto completo de descritores por sub-banda e gerar a matriz core de features.

Capítulo 2

Conjunto completo de descritores

Para cada sub-banda extraída pela WPT, calcularemos 9 descritores: Energia, RMS, Zero-Crossing Rate (ZCR), Média do envelope, Desvio padrão do envelope, Curtose do envelope, Entropia, Fator de forma e Fator de crista.

```
import numpy as np
from scipy.stats import kurtosis
from scipy.signal import hilbert

def calc_energia(sinal):
    return np.sum(sinal**2)
```

```
def calc_rms(sinal):
    return np.sqrt(np.mean(sinal**2))
```

```
def calc_zcr(sinal):
    return np.sum(np.diff(np.sign(sinal)) != 0) / len(sinal)
```

```
def envelope_sinal(sinal):
    return np.abs(hilbert(sinal))
```

```
def calc_env_media(sinal):
    return np.mean(envelope_sinal(sinal))
```

```
def calc_env_std(sinal):
    return np.std(envelope_sinal(sinal))
```

```
def calc_env_curtose(sinal):
    return kurtosis(envelope_sinal(sinal), fisher=True)
```

Capítulo 3

Entropia do sinal

A entropia de Shannon quantifica a incerteza ou complexidade de um sinal.

```
def calc_entropia(sinal):
    # normaliza
    sinal_norm = sinal ** 2
    s_sum = np.sum(sinal_norm)
    if s_sum == 0: return 0.0
    p = sinal_norm / s_sum
    p = p[p > 0]
    return -np.sum(p * np.log2(p))
```

Capítulo 4

Fator de forma

O fator de forma ê a razão entre o RMS e a média absoluta.

```
def calc_fator_forma(sinal):
    media_abs = np.mean(np.abs(sinal))
    if media_abs == 0: return 0.0
    return calc_rms(sinal) / media_abs
```

Capítulo 5

Fator de crista

O fator de crista ê a razão entre o valor de pico absoluto e o RMS.

```
def calc_fator_crista(sinal):
    rms = calc_rms(sinal)
    if rms == 0: return 0.0
    return np.max(np.abs(sinal)) / rms
```

Capítulo 6

Extrator completo

Vamos juntar tudo em uma funcão que extrai os 9 descritores para uma sub-banda.

```
def extrair_descritores_subbanda(sinal):
    return [
        calc_energia(sinal),
        calc_rms(sinal),
        calc_zcr(sinal),
        calc_env_media(sinal),
        calc_env_std(sinal),
        calc_env_curtose(sinal),
        calc_entropia(sinal),
        calc_fator_forma(sinal),
        calc_fator_crista(sinal)
    ]
```

Capítulo 7

Gerando a matriz de features

Aplicação a um nível 4 (16 sub-bandas). 16 * 9 = 144 features.

```
import pywt
def extrair_features_audio(audio, wavelet='db4', level=4):
    wp = pywt.WaveletPacket(data=audio, wavelet=wavelet, mode='
       symmetric', maxlevel=level)
    nodes = wp.get_level(level, 'freq')
    features = []
    for node in nodes:
        features.extend(extrair_descritores_subbanda(node.data))
    return np.array(features)
```

Capítulo 8

Salvando e carregando features

```
def salvar_features(X, y, filepath):
    np.savez_compressed(filepath, X=X, y=y)
```

```
def carregar_features(filepath):
    data = np.load(filepath)
    return data['X'], data['y']
```

Capítulo 9

Sanity checks

Verificações para evitar NaNs e Infinitos.

```
def sanity_check(X):
    if np.isnan(X).any():
        print("Aviso: NaN detectado!")
    if np.isinf(X).any():
        print("Aviso: Inf detectado!")
    print(f"Shape: {X.shape}")
```

Capítulo 10

Análise exploratória com PCA

O PCA reduz a dimensionalidade para 2D.

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

def plotar_pca(X, y):
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X)
    plt.scatter(X_pca[:,0], X_pca[:,1], c=y, alpha=0.5)
    plt.show()
```

Capítulo 11

Cache de features

Sempre cache as features baseando-se nas configurações.

```
import os
def get_cache_name(wavelet, level):
    return f"features_{wavelet}_L{level}.npz"
```

Capítulo 12

Brincadeiras

Vamos fazer alguns experimentos prãticos:

```
# exp 1
```

```
# exp 2
```

```
# exp 3
```

```
# exp 4
```

```
# exp 5
```

```
# exp 6
```

Capítulo 13

Checklist

- [x] Extração de energia - [x] Extração de entropia - [x] Fatores - [x] Vetor 144 features

Capítulo 14

Desafio final

Desafio

Otimize o código para rodar 10x mais rápido processando vários audios de uma vez.